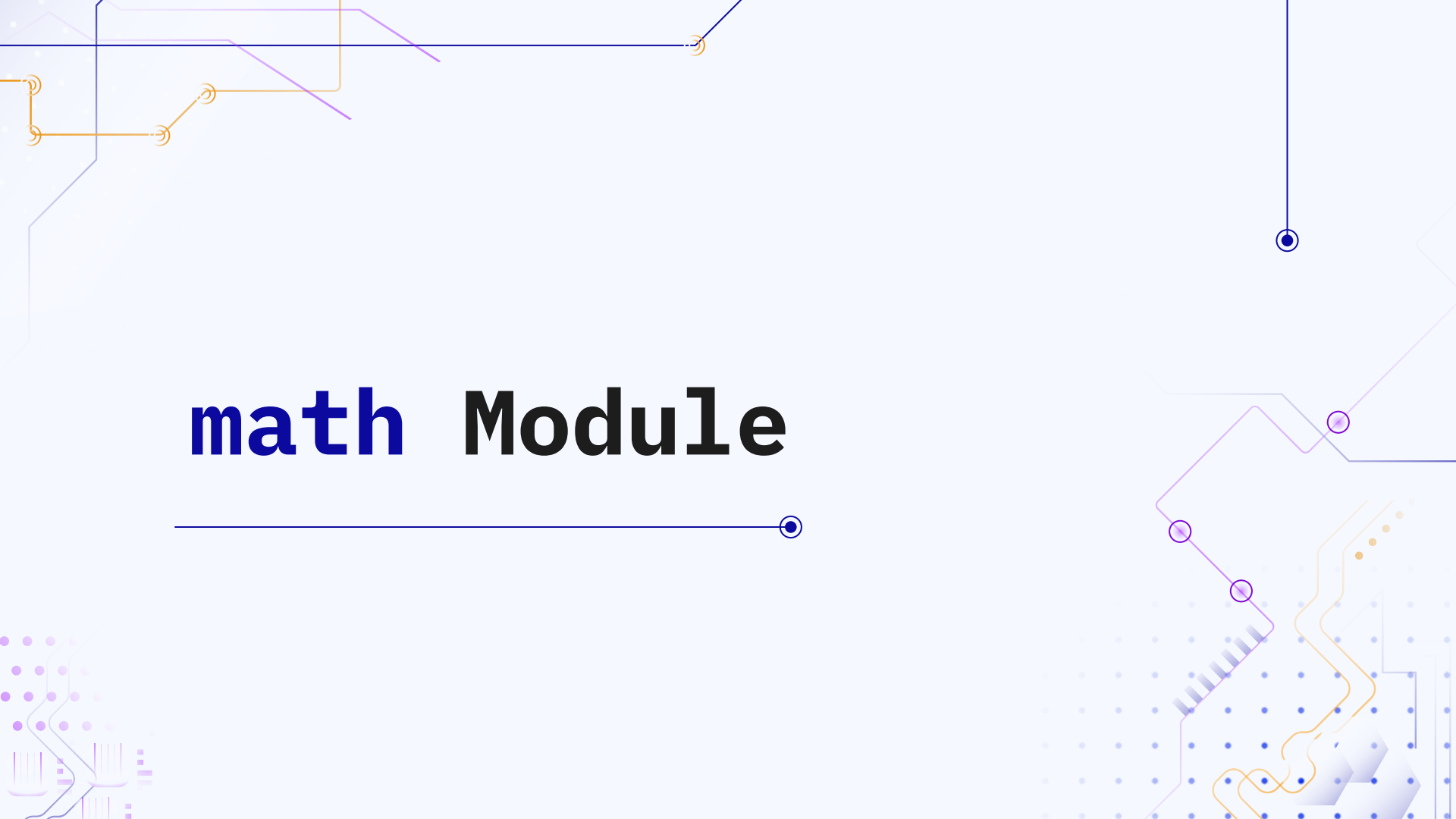




math Module

Introduction

Python's **math** module provides fast, reliable mathematical functions implemented close to the system level (C).

Use it when you need:

- Square root, powers, logs
- Trigonometry (sin, cos, tan, radians↔degrees)
- Rounding helpers like ceil, floor
- Factorial, gcd, combinations
- Constants like π

Works on real numbers (floats/ints).

For complex numbers, you use **cmath**.

For heavy matrix/math/science, you typically use **numpy**.

Constants (commonly used)

math.pi $\rightarrow \pi$ (pi) = 3.141592653589793

math.e $\rightarrow e$ (Euler's number) = 2.718281828459045

math.tau $\rightarrow 2\pi$ = 6.283185307179586

math.inf $\rightarrow +\text{infinity} = \text{inf}$

math.nan $\rightarrow$ Not-a-Number (invalid/undefined result) = nan

Rounding

Python

```
import math

# math.floor(x) = Largest integer  $\leq x$ 
math.floor(3.9)    # 3
math.floor(-3.1)   # -4

# math.ceil(x) = Smallest integer  $\geq x$ 
math.ceil(3.1)     # 4
math.ceil(-3.9)    # -3

# math.trunc(x) = Drops decimal part (towards zero)
math.trunc(3.9)    # 3
math.trunc(-3.9)   # -3
```

Powers, roots, and distance

Python

```
import math

# math.sqrt(x) = Square root (x must be ≥ 0)
math.sqrt(49)      # 7.0

# math.pow(a, b) = Power (returns float)
math.pow(2, 3)     # 8.0

# math.fsum(iterable) = More accurate sum for floats than sum()
math.fsum([0.1, 0.1, 0.1])  # 0.30000000000000004

# math.hypot(x, y, ...) = Euclidean distance
math.hypot(3, 4)   # 5.0
```

Logs and exponentials

Python

```
import math

# math.exp(x) = e ^ x
math.exp(1)      # 2.718...

# math.log(x, base)
math.log(8, 2)    # 3.0
math.log2(8)      # 3.0
math.log(100, 10) # 2.0
math.log10(100)   # 2.0
```

Rule: x must be > 0 , otherwise error.

Trigonometry

Python trig functions use **radians**, not degrees.

Convert degrees $\leftrightarrow$ radians

- `math.radians(deg)`
- `math.degrees(rad)`

Main trig functions

- `sin`, `cos`, `tan`
- Inverse: `asin`, `acos`, `atan`

If you pass degrees directly into `sin/cos/tan`, Python will treat that number as radians, so you'll get the "wrong" value

Python

```
import math

angle = math.radians(90)
print(math.sin(angle))  # 1.0 (approximately)
```

Combinatorics & number theory

Python

```
import math

# math.factorial(n) = n! for non-negative integers
math.factorial(5)      # 120

# math.gcd(a, b, ...) = Greatest common divisor
math.gcd(12, 18)       # 6

# math.lcm(a, b, ...) = Least common multiple
math.lcm(12, 18)       # 36

# math.comb(n, k) = Number of ways to choose k from n
math.comb(5, 2)        # 10

# math.perm(n, k=None) = Number of permutations (arrangements)
math.perm(5, 2)        # 20
```


Floating-point comparisons

Python

```
import math

# math.isclose(a, b, rel_tol=..., abs_tol=...)
math.isclose(0.1 + 0.2, 0.3)    # True
```

Due to floating-point representation, direct equality can fail:

```
print(0.1 + 0.2 == 0.3)    # False (often)
```

Checking for NaN / Infinity

- `math.isnan(x)`
- `math.isinf(x)`
- `math.isfinite(x)` → not NaN and not infinity

Python

```
import math

math.isfinite(10)      # True
math.isfinite(math.inf) # False
```

Common errors

1. Using degrees directly in **sin/cos/tan**

Fix: use **math.radians()**

2. Square root of negative number

math.sqrt(-1) gives error; use **cmath.sqrt(-1)** for complex

3. Expecting integer from **math.pow**

math.pow(2, 3) returns **8.0**. Use **2**3** if you want **int**.

4. Comparing **floats** with **==**

Use **math.isclose()**.

Recap

`math.pi`

`math.e`

`sqrt`

`pow`

`hypot`

`floor`

`ceil`

`trunc`

`fsum`

`log`

`log10`

`log2`

`exp`

`radians`

`degrees`

`sin`

`cos`

`tan`

`factorial`

`gcd`

`lcm`

`comb`

`perm`

`isclose`

`isfinite`

`isnan`

`isinf`

WATCH

Level up your coding with each episode in this focused Python series.



Next Video!

**collections
Module**

